

JusticeFlow

AI-Powered Police Case & Evidence Management System

Submitted By: Muhammad Furqan Sheikh (24K-0527),
Muhammad Abu Bakar (24K-0826),
Abdullah Muaz Abbasi (24K-0764)

Course: Artificial Intelligence (AI)

Instructor: Mr. Atif Luqman

Submission Date: March 6, 2026

1. Project Overview

Project Topic

JusticeFlow is an AI-powered decision support system embedded within a Police Case & Evidence Management platform. It deploys three independent machine-learning agents that assist law enforcement officers in detecting crime hotspots, prioritizing open cases, and optimally balancing officer workloads — all without replacing human authority.

Project Context & Scope

The three ML agents described in this proposal are the **AI layer of a larger, four-course integrated project**. JusticeFlow is being developed simultaneously across DBMS, SDA, AI, and OS — each course solving a distinct, non-overlapping problem that naturally arises in legal-grade software. The table below summarizes each course’s role in the unified system; the remainder of this document focuses exclusively on the AI contribution.

Course	Core Responsibility	Key Contribution to JusticeFlow
DBMS	Data Integrity & Audit	Fully normalized PostgreSQL schema (3NF), ACID transactions, immutable audit triggers, stored procedures, and role-restricted views ensuring legal-grade data correctness
SDA	Architecture & Patterns	Five justified design patterns (Strategy, Chain of Responsibility, Observer, Factory, Singleton), full UML suite (Use Case, Class, Sequence, State, Activity), and layered C++ application architecture
AI ★ This Proposal	Intelligent Decision Support	Three ML agents: DBSCAN crime hotspot detection, Random Forest case prioritization with SHAP explainability, and Hungarian Algorithm workload balancing — the focus of this document
OS	Process Isolation & Safety	Privilege daemon (justice_authd), inotify-based file integrity monitor, POSIX named semaphores for concurrent case updates, and a custom job scheduler for automated audit rotation and ML retraining

Note to Instructor: The DBMS, SDA, and OS layers are documented in separate course-specific submissions. This proposal covers only the AI layer. All three ML agents are independently runnable and verifiable against the shared PostgreSQL analytics schema without requiring the other layers to be present.

Objective

The primary goal is to demonstrate real-world application of AI techniques — unsupervised clustering, supervised classification, and combinatorial optimization — in a mission-critical legal-grade environment. Concretely, the system aims to:

- Identify the top 5 high-crime geographic zones from historical case data using DBSCAN clustering
- Rank open cases by urgency using a Random Forest model with SHAP-based explainability
- Assign cases to officers optimally via the Hungarian Algorithm, minimizing workload variance
- Surface all AI predictions transparently on the officer dashboard, with full audit trails

2. System Description

Original Problem Background

Law enforcement agencies handling large volumes of cases face three recurring operational bottlenecks:

- Manual prioritization: Investigators subjectively rank cases without data-driven guidance, risking high-severity crimes being deprioritized.
- Blind patrol deployment: Without geographic crime pattern analysis, patrol resources are distributed uniformly rather than concentration-optimized.
- Uneven workloads: Case assignments are often made ad hoc, resulting in some officers being overloaded while others are underutilized.

These problems compound in high-volume environments where hundreds of FIRs are filed daily and real-time decision support is critical.

Innovations Introduced

JusticeFlow introduces three AI-driven innovations that directly address each bottleneck:

1. Crime Hotspot Analyzer (DBSCAN)

Applies density-based spatial clustering to geo-tagged case records, automatically discovering high-density crime zones without requiring a predefined number of clusters. Output is a live heat map with patrol recommendations, updated on every new FIR.

2. Case Priority Recommender (Random Forest)

Trains a supervised ensemble classifier on 5,000+ synthetic cases labeled by severity. Features include crime type, time elapsed, evidence count, suspect status, and zone classification. SHAP (SHapley Additive exPlanations) values accompany every prediction, making each recommendation auditable and transparent to human reviewers.

3. Officer Workload Balancer (Hungarian Algorithm)

Formulates case-to-officer assignment as a minimum-cost bipartite matching problem. The Hungarian Algorithm solves this in polynomial time, producing an assignment that minimizes the standard deviation of workload across the officer pool — provably optimal given current constraints.

3. AI Approach and Methodology

Algorithms and Techniques

AI Agent	Algorithm	Output
Crime Hotspot Analyzer	DBSCAN (Unsupervised Clustering)	Heat map of top 5 crime zones with patrol recommendations
Case Priority Recommender	Random Forest (Supervised Classification)	Ranked case queue with SHAP-based feature explanations
Officer Workload Balancer	Hungarian Algorithm (Combinatorial Optimization)	Optimal case-to-officer assignments minimizing workload variance

Additional Techniques

- SHAP (SHapley Additive exPlanations): Provides per-prediction feature importance for the Random Forest model, satisfying legal explainability requirements.
- Synthetic Data Generation: 5,000+ cases generated with realistic statistical correlations; validated against UCI ML Repository crime datasets to ensure representativeness.
- Read-only process isolation: All AI agents run as separate read-only OS processes against the database analytics schema — they cannot modify operational data.
- Analytics schema: AI outputs are written to a dedicated analytics schema, decoupled from the operational DBMS layer, preserving ACID integrity of the core database.

4. System Rules and Mechanics

System Workflow

The end-to-end flow for a new FIR triggers all three AI agents in sequence:

Step	Component	What Happens
1	Input Layer	Officer files a new FIR; the system validates rank and filing rights via the authorization chain
2	Database Trigger	Atomic INSERT into Cases table; audit trigger fires and new case is persisted to the analytics schema
3	DBSCAN Agent	Crime Hotspot Analyzer re-clusters all cases to update the live heat map with patrol recommendations
4	Random Forest Agent	Case Priority Recommender scores the new case and inserts SHAP-explained priority ranking into dashboard
5	Hungarian Agent	Workload Balancer re-runs optimal assignment across all open cases and available officers
6	Officer Dashboard	Officer sees FIR number, crime zone classification, priority score, and updated duty assignment

Output and Success Conditions

The system succeeds when all three agents produce verifiable, explainable outputs within acceptable accuracy thresholds:

- DBSCAN produces a stable heat map where $\geq 70\%$ of flagged zones match ground-truth high-crime areas in the validation dataset.
- Random Forest achieves $\geq 75\%$ classification accuracy on the held-out test split; every prediction includes a SHAP explanation.
- Hungarian Algorithm produces a case assignment where officer workload standard deviation is provably lower than the baseline (round-robin) assignment.

Verification Criteria

Instructors can independently verify AI contributions by:

- Running Python scripts in isolation against the analytics schema — no dependency on the OS or SDA layers.
- Inspecting scikit-learn model serialization files and SHAP output logs.
- Querying the analytics schema tables directly to confirm AI output persistence.
- Reviewing the training/test split methodology and confusion matrices in the evaluation notebook.

5. Libraries and Tools

Core AI Libraries

- scikit-learn: DBSCAN (`sklearn.cluster.DBSCAN`), Random Forest (`sklearn.ensemble.RandomForestClassifier`), Hungarian Algorithm via `scipy.optimize.linear_sum_assignment`
- SHAP: Model explainability and feature importance visualization (`shap.TreeExplainer` for Random Forest)
- NumPy & Pandas: Data manipulation, feature engineering, and synthetic dataset generation
- Matplotlib / Seaborn: Heat map visualization and performance metric plots

Data & Integration Tools

- PostgreSQL (`psycopg2`): Read-only connections to the analytics schema for fetching case data
- Faker + NumPy random: Synthetic dataset generation with realistic crime-domain distributions
- Joblib: Model serialization and caching for Random Forest

Development & Validation

- Jupyter Notebook: Model training, evaluation, and SHAP visualization
- Git: Version control and submission
- UCI ML Repository: External crime datasets for validation benchmarking
- We will be creating our a dataset matching our schema as the main dataset

Success Metrics Summary

Metric	Description	Target
Hotspot Detection Precision	Correct identification of high-crime zones by DBSCAN	> 70%
Case Priority Accuracy	Random Forest classification accuracy on test set	> 75%
Workload Variance	Standard deviation of case load across officers post-assignment	Minimized
SHAP Explainability	All predictions accompanied by feature-importance explanation	100%
Training Dataset Size	Synthetic cases generated with realistic correlations	5,000+

6. References

Ester, M., Kriegel, H. P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. KDD-96 Proceedings, 226–231.

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. Naval Research Logistics Quarterly, 2(1–2), 83–97.

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

UCI Machine Learning Repository — Crime and Communities Dataset. Available at: <https://archive.ics.uci.edu/ml/datasets/Communities+and+Crime>

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.